

CSC-696D, Takanori Fujiwara

Final project report.

Novel visualization techniques for git data.

Source Code: <https://github.com/rowan907/CSC-696D-Final-Project>.

Hosted project: <https://rowan907.github.io/CSC-696D-Final-Project>.

0.1 Problem Description

In this project we created several novel visual analytic systems for git data. The visualizations we propose will help developers quickly gain an understanding of git repositories. In particular, we seek to give users an understanding of how a repo changes over time. We will focus on a few key questions.

1. Who are the main contributors to a repository?
2. What were the development priorities for a repository at any given time?
3. Can we identify a time when AI-assisted development became popular on a given repository?
4. What is the best visualization for giving developers an understanding of the evolution of project structure over time?

0.2 Motivation

Developers often work with large amounts of code and are forced to quickly learn the customs and conventions of many repositories, this process can be very time-consuming. By giving developers tools for quickly visualizing the history of a repository, we can immediately give them an understanding of conventions, key contributors, and etiquette.

Many visualizations have been developed over the years to work with git data, in this project we developed a handful of new visualizations by first studying visualizations that others have developed, then either refining them to improve their visual clarity or augmenting them with additional data.

We developed 3 visualizations for this project.

1. A ‘stabilized’ Sediment Graph based on the work of Erik Bernhardsson. Bernhardsson proposes a unique way to understand the age of the contents of a repo, improved his designs by changing the stacking behavior so as rows are added we allow the graph to grow in the positive and negative y directions. This improves the stability of the

graph by keeping individual lines of sediment more horizontal. Reducing the visual interference caused by the sediment layers traveling diagonally up the graph, making there width harder to determine.

We also (based on feedback we received from our peers), added to new views to this visualization. In addition to dividing the layers of sediment by code age we also provided the option to divide the sediment by author, supporting both total lines of code by author and number of commits by author. These views may be an even better use case then code age, as authorship is generally a more important metric.

2. Word cloud style view that allows users to select many commits via a time window selection and see a word cloud of the commit messages. This will allow users to quickly see the development focuses of a repo at a particular time window. This is inspired by a view from the paper *Visual Analytics for Understanding Software Development History Through Git Metadata Analysis*. We seek to enhance their view by adding a time window selection allowing these clouds to be generated for any section of the commit history. We also provide users the ability to click a word from the word cloud and highlight the commits that include that word. Allowing users to quickly see what changes dealt with certain topics.
3. A grouping-based visualization based on a force directed graph that clusters files or directories that are often modified together, this will help users understand which parts of the codebase are most tightly coupled. We animated this view to show the evolution of the repo over time and coupled it with a timeline view allowing users to scrub to particular sections of the animation.

0.3 Related Research Papers

- Githru Visual Analytics for Understanding Software Development History Through Git Metadata Analysis — This work shows an all-in-one visual analytic tool for examining git history and provides a lot of great background on git. Many techniques here may be useful; in particular, the paper provides a lot of detail on the technical aspects of creating visualizations with git metadata. We also took inspiration from this paper for our second proposed visualization as discussed above.
- Git-Truck: Hierarchy-Oriented Visualization of Git Repository Evolution — Git-Truck is a hierarchical file-organization-oriented visualization tool that represents git repositories as different treemaps. Each treemap has its nodes sized by the number of lines of code in the repository, and the color of its nodes can represent a variety of interesting factors such as number of commits, top contributors, or even the last change day.

This is showing the current structure of a repository, rather than the temporal focus we have in this project. Mainly, we emphasize the importance of seeing the historical development of features in a repository.

- **ChangePrism: Visualizing the Essence of Code Changes** — This paper gives another approach to visualizing code changes that relies on semantic analysis. Though this paper's focus is more on visualizing individual commits and discrete changes rather than the type of broad analysis of larger repositories we did, it gives some good background on existing visual analytic techniques.

0.4 Technical Approach

Our technical approach is at a high level

1. Collect `.git` files containing git meta-data from repos we wish to study.
2. Use a pre-processing script to execute git commands to query this data and create a `.json` file for each repo containing all commits we want to analyze and any useful meta-data like branch, parent, files changed etc.
3. Load our `.json` files at run time when the users selects a particular repo they want to analyze.
4. Build our visualizations from the processed data.

Our pre-processing script lives inside the project (`app/scripts/preprocessgit.mjs`) and can be triggered manually with `npm run preprocess:git`. The script uses JavaScript's `exec` function to run git with the flag `--git-dir=[.git file]`, this command allows git to behave for most commands as if it's inside the project we're targeting despite only having access to the metadata. This saves us having to pull the whole project we wish to study.

We then run a `git log` to get all commit hashes across all branches (for performance reasons we cap this at the 4500 most recent commits when more are available). Finally, we run git commands on every individual commit to get information like predecessor and branch. The information we pull from each commit is bellow and is used to support all three visualizations.

```
type Commit {  
  hash: string;  
  parents: Commit[];  
  children: Commit[];  
  author: string;  
  date: string;  
  subject: string;  
  branch: string | null;  
  merge?: string | null;  
  files?: CommitFile[];  
}  
  
type CommitFile {  
  additions: number;  
  deletions: number;  
  path: string;  
}
```

We then include all json files in the `public` directory of our frontend app we configure React to serve all files in this directory, so that at runtime we can query them as needed. We do this because these files are fairly large and not bundling all of them with our app directly reduces load time.¹

To speed up implementation we used a few AI tools primary Claude Haiku/Sonnet/Opus 4.x, as well as Qwen 3.6 running locally for smaller changes and refactors.

0.5 Technical Challenges

We initially tried to get all the information we'd need from the git log command however this involved manually parsing commit messages to try and infer things like merges which was not completely accurate. In the end we settled on the approach of getting the commit hashes from the log and running a git command for every commit to get the additional data we needed. This approach was much slower but more reliable, and since the processing script only needs to run once the slowdown was acceptable.

0.6 Selected Dataset(s)

We selected 4 medium size repos to examine in this project.

- Flask
- Click
- Httpx

¹If you're curious you can see the raw json files by navigating to https://rowan907.github.io/CSC-696D-Final-Project/flask_commits.json

- Requests

0.7 Expected results and evaluation methodology

In this project, we seek to create a web application to demonstrate and explain our visual analytic systems. It will feature the visualizations we've created, a short text description of the visualizations—explaining works inspiring the visualization, the types of analytical questions the visualization seeks to answer and the strengths and weaknesses of our approach. The app will also feature a selector that will allow you to view any git repository with any of the aforementioned visualization techniques. We will then gather feedback from professional software engineers on the usefulness of our visualizations and applicability of our tools to their current development workflow.